01 PRACTICAL 1

ASSIGNMENT 3 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

from tensorflow.keras import models, layers, optimizers

# LOADING DATASET
df = pd.read_csv("creditcard.csv")
df.head()

print(df.shape)
# print(df.info())
print(df['Class'].isnull().sum().sum())

# SEPARATING X AND Y
X = df.drop(columns=["Class", "Time"])
y = df["Class"]

# SPLIT AND SCALE THE DATA
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# SEPARATING NORMAL AND FRAUD TRANSACTION
X_train = X_train[y_train == 0]

# DEFINING AUTOENCODER ARCHITECTURE

#ENCODER
encoder = models.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu")
])

# DECODER
decoder = models.Sequential([
    layers.Input(shape=(8,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(X_train.shape[1], activation="sigmoid")
])

# AUTOENCODER
model = models.Sequential([encoder, decoder])

model.summary()
encoder.summary()
decoder.summary()
```

```
# COMPILE THE MODEL
model.compile(
    optimizer = 'adam',
    loss = "mse",
    metrics = ["mae"]
)

# TRAIN THE MODEL
history = model.fit(
    X_train, X_train,
    epochs = 10,
    batch_size = 256,
    validation_split = 0.1
)

# CALCULATE RECONSTRUCTION ERROR
train_predictions = model.predict(X_train)

train_errors = np.mean(
    np.square(X_train - train_predictions), axis = 1
)

# SETTING THE THRESHOLD TO 95TH PERCENTILE
threshold = np.percentile(train_errors, 95 )

print("Threshold: ", threshold)

# DETECT FRAUDENT TRANSACTIONS
test_predictions = model.predict(X_test)

test_errors = np.mean(
    np.square(X_test - test_predictions), axis = 1
)

y_pred = (test_errors > threshold).astype(int)

print("Reconstruction Errors:")
print(test_errors[:10])

print("Predicted Classes:")
print(y_pred[:10])

y_pred

# EVALUATE MODEL PERFORMANCE
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

plt.hist(test_errors, bins=5)
plt.axvline(threshold, color='red', linestyle='--')
plt.xlabel("Reconstruction Error")
plt.ylabel("Number of Transactions")
plt.show()
```

02 PRACTICAL 2

ASSIGNMENT 3 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

from tensorflow.keras import models, layers, optimizers

# LOADING DATASET
df = pd.read_csv("ecg.csv", header = None)
df.head()

print(df.shape)
print(df.iloc[:, -1].isnull().sum().sum())

# SEPARATE X AND Y
x = df.iloc[:,:-1].values
y = df.iloc[:,-1].values

# NORMALIZE DATA
scaler = StandardScaler()
x = scaler.fit_transform(x)

# SPLIT THE DATASET
X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size = 0.2, random_state = 42
)

# SEPARATING NORMAL AND ABNORMAL ECG
X_train = X_train[y_train == 0]

# DEFINING AUTOENCODER ARCHITECTURE

# ENCODER
encoder = models.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu")
])

# DECODER
decoder = models.Sequential([
    layers.Input(shape=(8,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(X_train.shape[1], activation="sigmoid")
])

# AUTOENCODER
model = models.Sequential([encoder, decoder])

model.summary()
encoder.summary()
decoder.summary()

# COMPILE THE MODEL
```

```
model.compile(
    optimizer = 'adam',
    loss = "mse",
    metrics = ["mae"]
)

# TRAIN THE MODEL
history = model.fit(
    X_train, X_train,
    epochs = 10,
    batch_size = 256,
    validation_split = 0.1
)

# CALCULATE RECONSTRUCTION ERROR
train_predictions = model.predict(X_train)

train_errors = np.mean(
    np.square(X_train - train_predictions), axis = 1
)

# SETTING THE THRESHOLD TO 95TH PERCENTILE
threshold = np.percentile(train_errors, 90 )

print("Threshold: ", threshold)

# DETECT FRAUDENT TRANSACTIONS
test_predictions = model.predict(X_test)

test_errors = np.mean(
    np.square(X_test - test_predictions), axis = 1
)

y_pred = (test_errors > threshold).astype(int)

print("Reconstruction Errors:")
print(test_errors[:20])

print("Predicted Classes:")
print(y_pred[:20])

# EVALUATE MODEL PERFORMANCE
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

03 PRACTICAL 3

ASSIGNMENT 3 • CODE REVISION SHEET

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

from tensorflow.keras import models, layers

data = pd.read_csv("energy consumption.csv")

print(data.head())
print(data.shape)

# SEPARRATE x AND Y
X = df.drop(columns = ['Timestamp', 'Anomaly_Label'])
y = df['Anomaly_Label']

# PREPROCESSING
data = data.fillna(data.mean(numeric_only = True))

x = data.drop(['Timestamp', 'Anomaly_Label'], axis=1)
y = data['Anomaly_Label']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42, stratify=y
)

# KEEP ONLY NOMRMAL DATA FOR TRAINING
normal = x_train[y_train==0]

# Normalize data
scaler = StandardScaler()
normal = scaler.fit_transform(normal)
x_test = scaler.transform(x_test)

# Split normal data into training and validation
x_train, x_val = train_test_split(
    normal, test_size=0.1, random_state=42
)

# Reshape data for LSTM
x_train = x_train.reshape(-1, x_train.shape[1], 1)
x_val = x_val.reshape(-1, x_val.shape[1], 1)
x_test = x_test.reshape(-1, x_test.shape[1], 1)

# Step 4: Build LSTM Encoder and Decoder

encoder = models.Sequential([
    layers.Input(shape=(x_train.shape[1], 1)),
    layers.LSTM(16)
])

decoder = models.Sequential([
    layers.Input(shape=(16,)),
    layers.RepeatVector(x_train.shape[1]),
    layers.LSTM(16, return_sequences=True),
    layers.TimeDistributed(layers.Dense(1))
])

model = models.Sequential([encoder, decoder])
```

```
encoder.summary()
decoder.summary()
model.summary()

# Step 5: Compile model
model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

# Step 6: Train model
model.fit(
    x_train, x_train,
    validation_data=(x_val, x_val),
    epochs=20,
    batch_size=512
)

# Step 7: Calculate threshold
train_pred = model.predict(x_val)

train_error = np.mean(
    np.square(x_val - train_pred), axis=(1, 2)
)

threshold = np.percentile(train_error, 99)

print("Threshold:", threshold)

# Step 8: Detect anomalies
test_pred = model.predict(x_test)

test_error = np.mean(
    np.square(x_test - test_pred), axis=(1, 2)
)

prediction = (test_error > threshold).astype(int)

# Step 9: Evaluate model
print("Confusion Matrix:")
print(confusion_matrix(y_test, prediction))

print("\nClassification Report:")
print(classification_report(y_test, prediction))

print("Number of anomalies detected:", prediction.sum())
```

04 PRACTICAL 4

ASSIGNMENT 3 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

from tensorflow.keras import models
from tensorflow.keras import layers

# LOADING THE DATASET
data = pd.read_csv("NASA TurboFan Engine.csv")

data.head()
# print(data.shape)

# PREPROCESSING AND SPLITTING X AND Y
data = data.fillna(data.mean(numeric_only = True))

X = data.drop("Fault_Label", axis=1).values
y = data["Fault_Label"].values

# TRAIN AND TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    random_state = 42,
    test_size = 0.2
)

# KEEP ONLY NORMAL DATA FOR TRAINING
X_train = X_train[y_train == 0]

# NORMALIZE THE DATA
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# SPLIT TRAIN DATA INTO TRAINING AND VALIDATION
X_train, x_val = train_test_split(
    X_train,
    test_size = 0.1,
    random_state = 42
)

# RESHAPE DATA FOR LSTM
X_train = X_train.reshape(-1, X_train.shape[1], 1)
X_val = x_val.reshape(-1, x_val.shape[1], 1)
X_test = X_test.reshape(-1, X_test.shape[1],1)

# DEFINING AUTOENCODER ARCHITECTURE

# Encoder
encoder = models.Sequential([
    layers.Input(shape=(X_train.shape[1],1)),
    layers.LSTM(16)
])

# Decoder
decoder = models.Sequential([
    layers.Input(shape=(16,)),
```

```
    layers.RepeatVector(X_train.shape[1]),
    layers.LSTM(16, return_sequences=True),
    layers.TimeDistributed(layers.Dense(1))
])

# Autoencoder
model = models.Sequential([encoder, decoder])

# COMPILE AND TRAIN THE MODEL
model.compile(
    optimizer = 'adam',
    loss = 'mse',
    metrics = ['mae']
)

history = model.fit(
    X_train,X_train,
    validation_data=(X_val, X_val),
    epochs = 11,
    batch_size = 256
)

# EVALUATE THE MODEL
test_loss, test_accuracy = model.evaluate(X_test, X_test)

print("Test loss : ", test_loss)
print("Test accuracy: " , test_accuracy)

# CALCULATING RECONSTRCUTION ERROR AND SETTING THRESHOLD
train_pred = model.predict(X_val)

train_error = np.mean(
    np.square(X_test - test_pred) , axis = (1, 2)
)

threshold = np.percentile(train_error, 95)

print("Threshold: ", threshold)

# DETECT ANOMALIES
test_pred = model.predict(X_test)

test_error = np.mean(
    np.square(X_test - test_pred), axis=(1,2)
)

predictions = ( test_error > threshold ).astype(int)

print("Acutal: ", y_test[:10])
print("Predicted: " , predictions[:10])

# EVALUATE THE MODEL
print("Confusion Matrix: ")
print(confusion_matrix(y_test, predictions))

print("Classification Report" )
print(classification_report(y_test, predictions))
```

05 PRACTICAL 5

ASSIGNMENT 3 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.metrics import confusion_matrix, classification_report

from tensorflow.keras import models, layers

# LOADING THE DATASET
train = pd.read_csv("sign_mnist_train.csv")
test = pd.read_csv("sign_mnist_test.csv")

train.head()
train.shape

# PREPROCESS THE DATA
X_train = train.drop("label", axis = 1)
y_train = train["label"]

X_test = test.drop("label", axis=1 )
y_test = test["label"]

# NORMALIZE PIXEL VALUES
X_train = X_train / 255.0
X_test = X_test / 255.0

# SELECTING LABEL 24 AS ANOMALY
X_train = X_train[y_train != 24]

y_test = (y_test == 24).astype(int)

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)

# DEFINING MODEL ARCHITECTURE
# ENCODER
encoder = models.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(128, activation = "relu"),
    layers.Dense(64, activation = "relu"),
    layers.Dense(32, activation = "relu"),
    layers.Dense(8, activation = "relu")
])

# DECODRE
decoder = models.Sequential([
    layers.Input(shape=(8,)),
    layers.Dense(32, activation = "relu"),
    layers.Dense(64, activation = "relu"),
    layers.Dense(128, activation = "relu"),
    layers.Dense(X_train.shape[1], activation = "sigmoid")
])

# AUTOENCODER
model = models.Sequential([
    encoder,
    decoder
])

encoder.summary()
decoder.summary()
```

```
model.summary()

# COMPILE AND TRAIN THE MODLE
model.compile(
    optimizer = 'adam',
    loss = 'mse',
    metrics = ['mae']
)

history = model.fit(
    X_train,
    X_train,
    epochs= 11,
    batch_size = 64,
    validation_split = 0.2
)

# CALCULATE RECONSTRUCTION ERROR
train_predictions = model.predict(X_test)

train_errors = np.mean(
    np.square(X_test - train_predictions), axis = 1
)

# SETTING THE THRESHOLD
threshold = np.percentile(train_errors, 95)

print("Threshold: ", threshold)

# DETECT ANOMALIES
test_predictions = model.predict(X_test)

test_errors = np.mean(
    np.square(X_test - test_predictions), axis = 1
)

y_pred = (test_errors > threshold ).astype(int)

print(test_errors[:20])
print(y_pred[:20])

# EVALUATE THE MODEL
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("Classification report")
print(classification_report(y_test, y_pred))
```